
Estácio

- **CAMPUS**: POLO CENTRO- PINDORETAMA-CE
- **CURSO**: Desenvolvimento FullStack
- **DISCIPLINA**: Tecnologias para desenvolvimento de soluções de big data
- **TURMA**: 2026.2
- **SEMESTRE LETIVO**: 2026
- **ALUNO**: João Victor Holanda Silva
- **MATRÍCULA**: 2024020102959

Trabalho Prático | DGT2823 Tecnologias para desenvolvimento de soluções de big data:

Objetivos da prática:

1. Descrever como ler um arquivo CSV usando a biblioteca Pandas (Python);

2. Descrever como criar um subconjunto de dados a partir de um conjunto existente usando a biblioteca Pandas (Python);

3. Descrever como configurar o número máximo de linhas a serem exibidas na visualização de um conjunto de dados usando a biblioteca Pandas (Python);

4. Descrever como exibir as primeiras e últimas “N” linhas de um conjunto de dados usando a biblioteca Pandas (Python); Descrever como exibir informações gerais sobre as colunas, linhas e dados de um conjunto de dados usando a biblioteca Pandas (Python)

1º Procedimento | Criando o arquivo data.csv

ID	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/0	110	130	4091
1	60	'2020/12/0	117	145	4790
2	60	'2020/12/0	103	135	3400
3	45	'2020/12/0	109	175	2824
4	45	'2020/12/0	117	148	4060
5	60	'2020/12/0	102	127	3000
6	60	'2020/12/0	110	136	3740
7	450	'2020/12/0	104	134	2533
8	30	'2020/12/0	109	133	1951
9	60	'2020/12/1	98	124	2690
10	60	'2020/12/1	103	147	3293
11	60	'2020/12/1	100	120	2507
12	60	'2020/12/1	100	120	2507
13	60	'2020/12/1	106	128	3453
14	60	'2020/12/1	104	132	3793
15	60	'2020/12/1	98	123	2750
16	60	'2020/12/1	98	120	2152
17	60	'2020/12/1	100	120	3000
18	45	'2020/12/1	90	112	NaN
19	60	'2020/12/1	103	123	3230
20	45	'2020/12/2	97	125	2430
21	60	'2020/12/2	108	131	3642
22	45	NaN	100	119	2820
23	60	'2020/12/2	130	101	3000
24	45	'2020/12/2	105	132	2460
25	60	'2020/12/2	102	126	3345
26	60	20201226	100	120	2500
27	60	'2020/12/2	92	118	2410
28	60	'2020/12/2	103	132	NaN
29	60	'2020/12/2	100	132	2800
30	60	'2020/12/3	102	129	3803
31	60	'2020/12/3	92	115	2430

Microatividade 1| Ler um arquivo CSV usando Pandas

1. Importar a biblioteca pandas import pandas as pd

2. Criar uma variável e ler o conteúdo do arquivo CSV

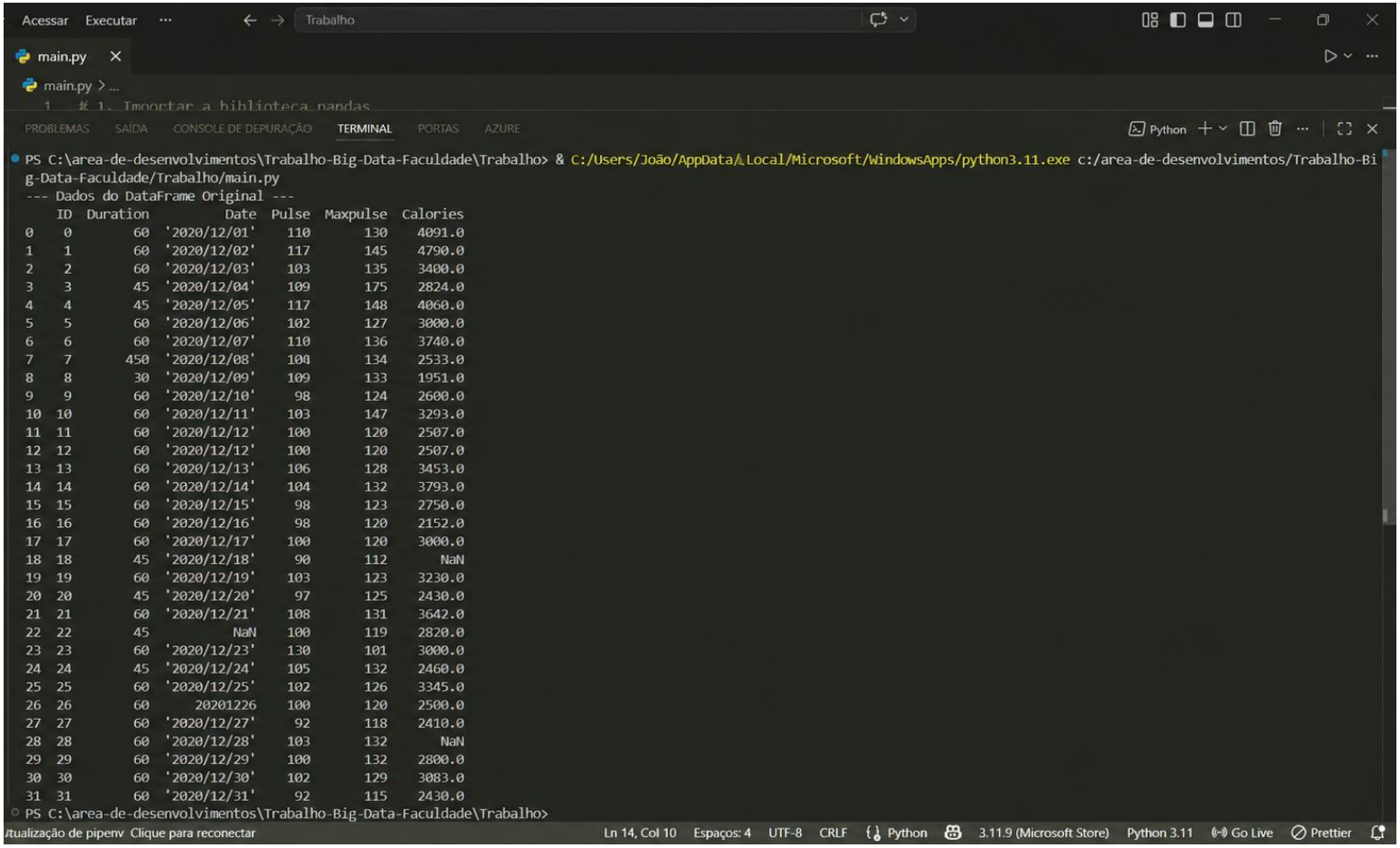
Parâmetros solicitados:

sep=';' (o arquivo usa ponto e vírgula)

engine='python' (garante compatibilidade com alguns formatos de CSV)

encoding='ISO-8859-1' ou 'utf-8' (opcional, vamos usar utf-8 por padrão) df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

3. Exibir os dados na tela print("--- Dados do DataFrame Original ---") print(df



- **import pandas as pd**: É a convenção padrão da comunidade. O pd facilita a chamada dos métodos.
- **df:** Abreviação de DataFrame, que é a estrutura de tabela do Pandas.

Microatividade 2 | Criar um subconjunto de dados

1. Importar a biblioteca pandas import pandas as pd

2. Criar uma variável e ler o conteúdo do arquivo CSV

Parâmetros solicitados: # sep=';' (o arquivo usa ponto e vírgula)

engine='python' (garante compatibilidade com alguns formatos de CSV)

encoding='ISO-8859-1' ou 'utf-8' (opcional, vamos usar utf-8 por padrão)

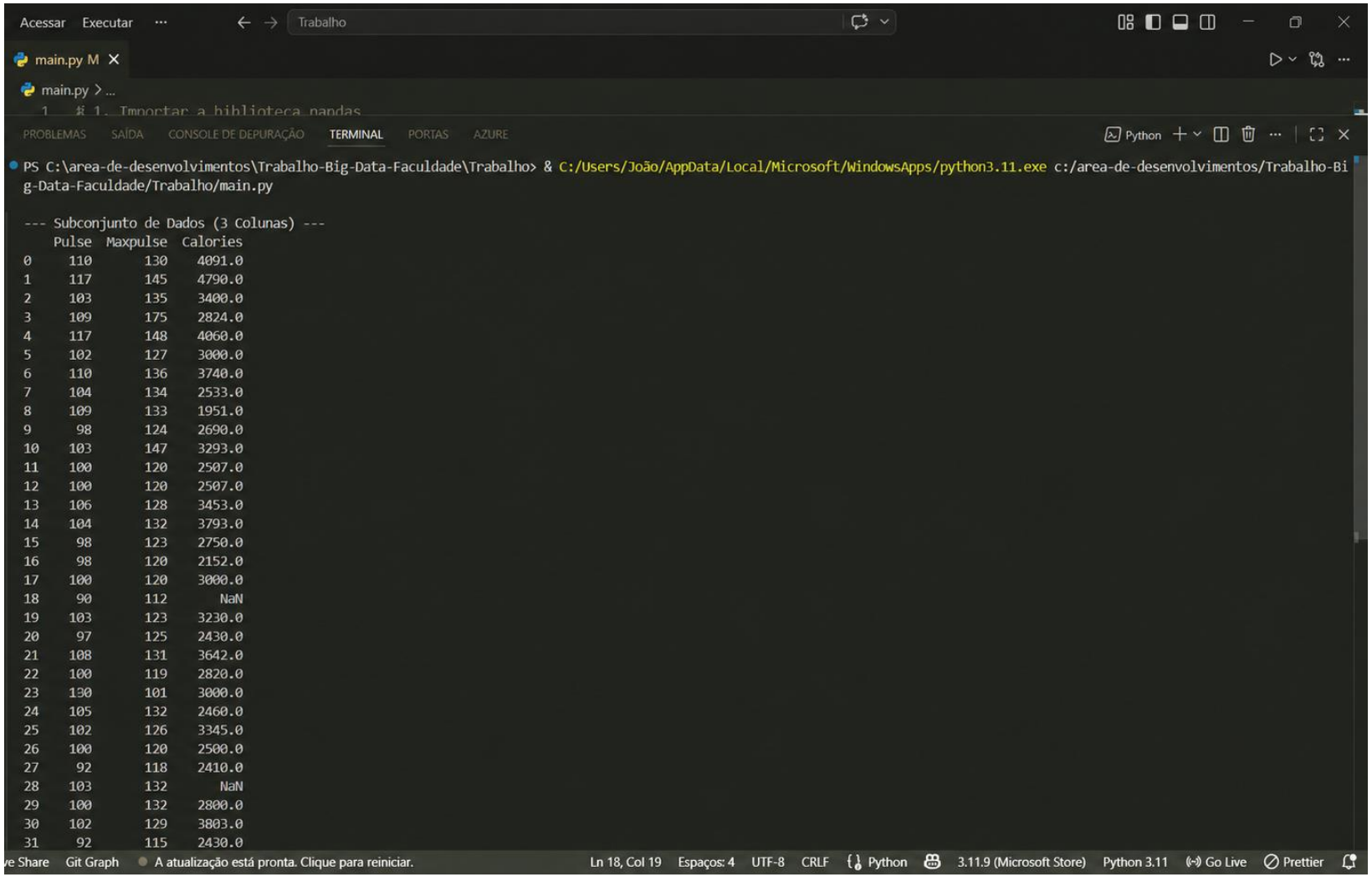
df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

1. Criar uma nova variável para o subconjunto

Selecionando

3 colunas: 'Pulse', 'Maxpulse' e 'Calories' df_reduzido = df[['Pulse', 'Maxpulse', 'Calories']]

2. Exibir o novo subconjunto print("\n--- Subconjunto de Dados (3 Colunas) ---") print(df_reduzido)



Explicação : Ao usar colchetes duplos [[...]], estamos passando uma lista de colunas para o DataFrame original, o que gera um novo DataFrame apenas com essas fatias de dados

Microatividade 3 | Configurar o número máximo de linhas exibidas

1. Importar a biblioteca pandas import pandas as pd

2. Criar uma variável e ler o conteúdo do arquivo CSV

Parâmetros solicitados:

sep=';' (o arquivo usa ponto e vírgula)

engine='python' (garante compatibilidade com alguns formatos de CSV)

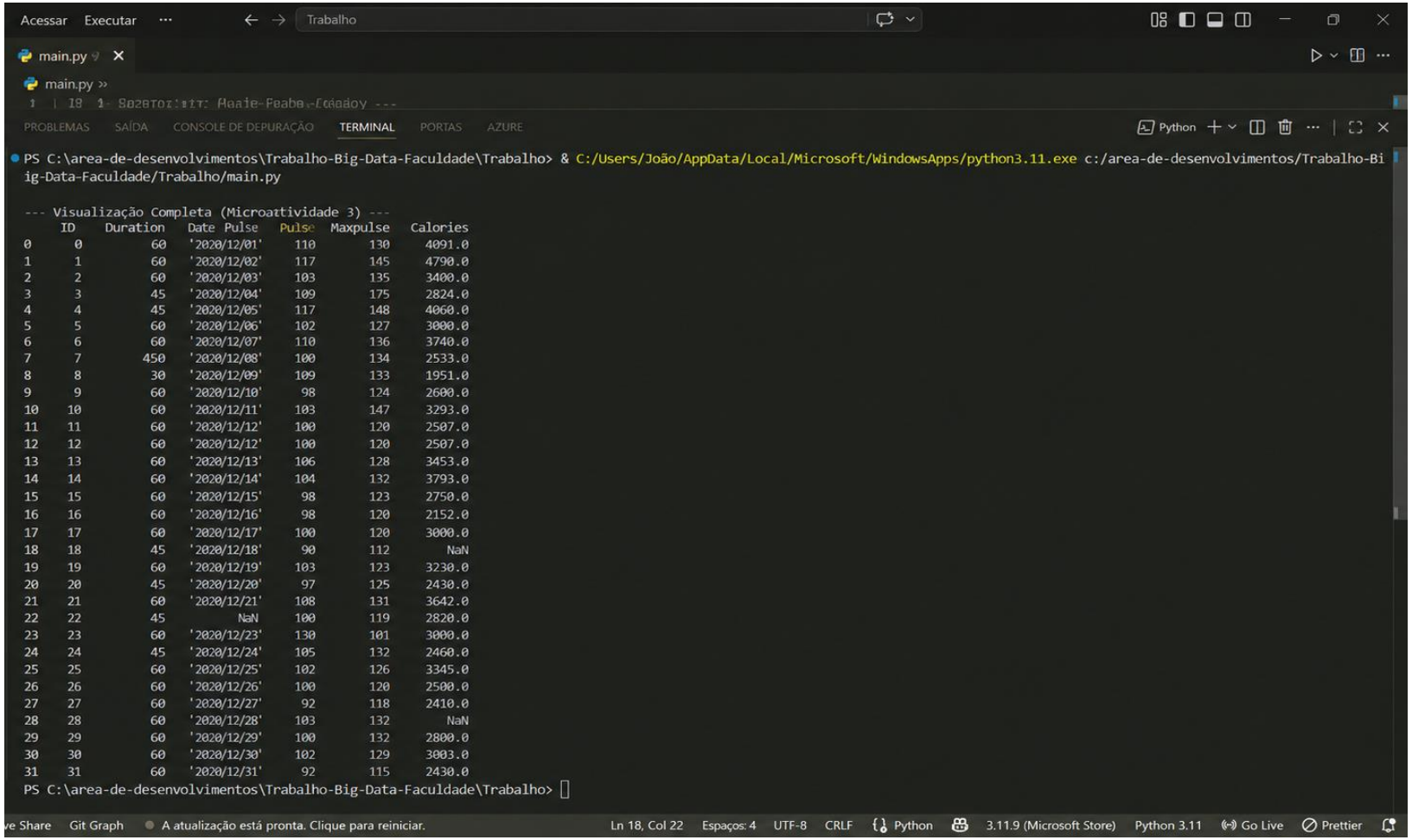
encoding='ISO-8859-1' ou 'utf-8' (opcional, vamos usar utf-8 por padrão)

df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

1. Definir o valor máximo de linhas para exibição como 9999 pd.options.display.max_rows = 9999

2. Imprimir o conjunto original usando o método to_string()

O to_string() transforma o DataFrame em uma string completa, sem resumos. print("\n--- Visualização Completa (Microatividade 3) ---") print(df.to_string())



Explicação : Por padrão, se um DataFrame for muito grande, o Pandas mascara as linhas do meio para não poluir a tela. Aqui, vamos forçá-lo a mostrar tudo usando as opções de configuração.

Microatividade 4 | Exibir as primeiras e últimas “N” linhas

1. Importar a biblioteca pandas import pandas as pd

2. Criar uma variável e ler o conteúdo do arquivo CSV

Parâmetros solicitados:

sep=';' (o arquivo usa ponto e vírgula)

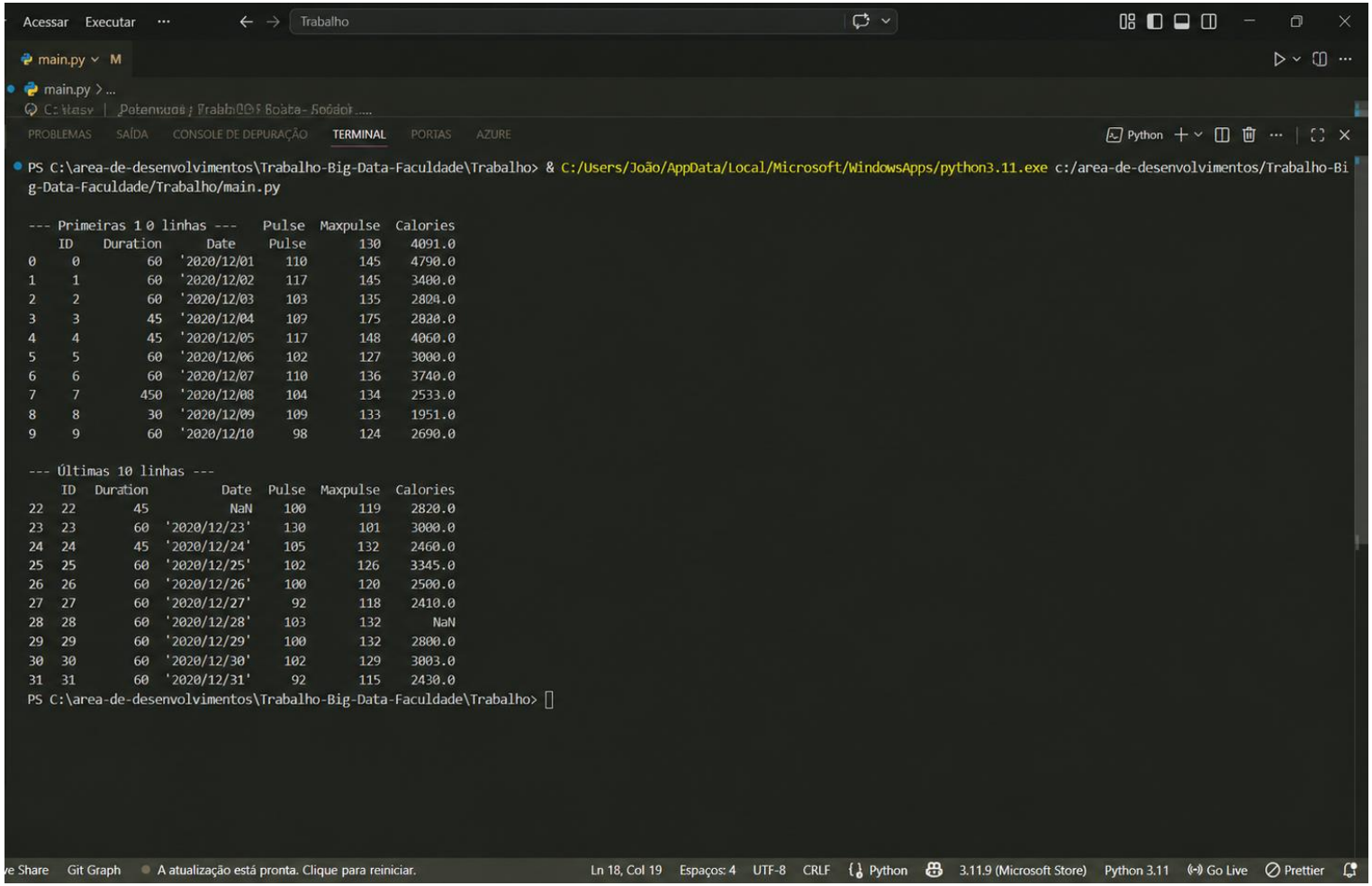
engine='python' (garante compatibilidade com alguns formatos de CSV) # encoding='ISO-8859-1' ou 'utf-8' (opcional, vamos usar utf-8 por padrão)

df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

1. Definir o valor máximo de linhas para exibição como 9999 pd.options.display.max_rows = 9999

2. Imprimir o conjunto original usando o método to_string()

O to_string() transforma o DataFrame em uma string completa, sem resumos. print("\n--- Visualização Completa (Microatividade 4) ---") print(df.to_string())



Explicação: Em Big Data, raramente olhamos o arquivo inteiro. Geralmente, usamos o head (cabeça) e o tail (cauda) para verificar se o início e o fim dos dados fazem sentido.

Microatividade 5 | Informações gerais sobre o conjunto de dados

1. Importar a biblioteca pandas import pandas as pd

2. Criar uma variável e ler o conteúdo do arquivo CSV

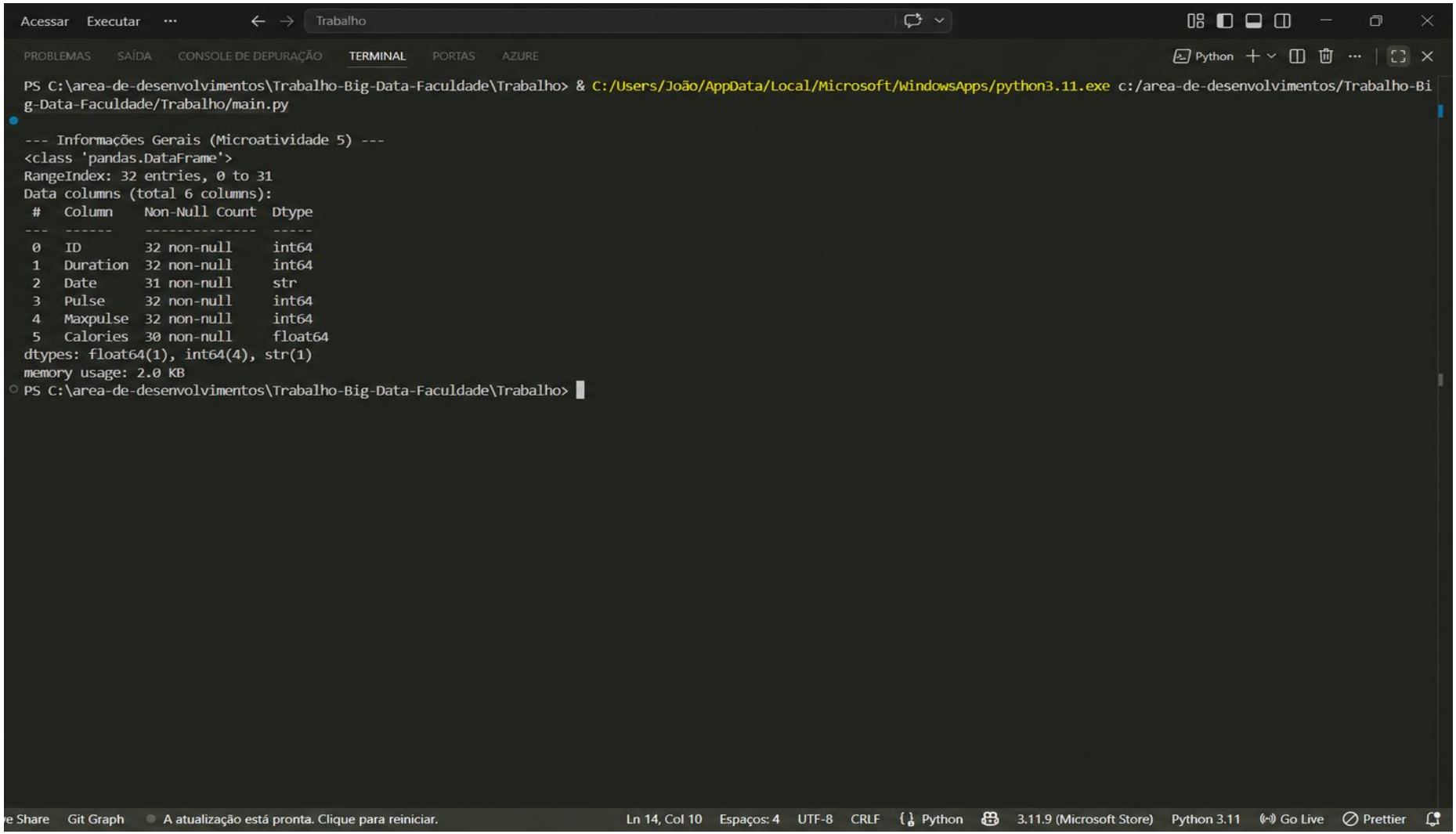
Parâmetros solicitados:

sep=';' (o arquivo usa ponto e vírgula)

engine='python' (garante compatibilidade com alguns formatos de CSV) # encoding='ISO-8859-1' ou 'utf-8' (opcional, vamos usar utf-8 por padrão)

```
df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')
```

1. Imprimir informações gerais do DataFrame print("\n--- Informações Gerais (Microatividade 5) ---") df.info()



Explicação : Esta etapa é muito importante para um Analista de Dados. O método .info() é o "raio-x" do arquivo: total de linhas, colunas, nulos e memória.

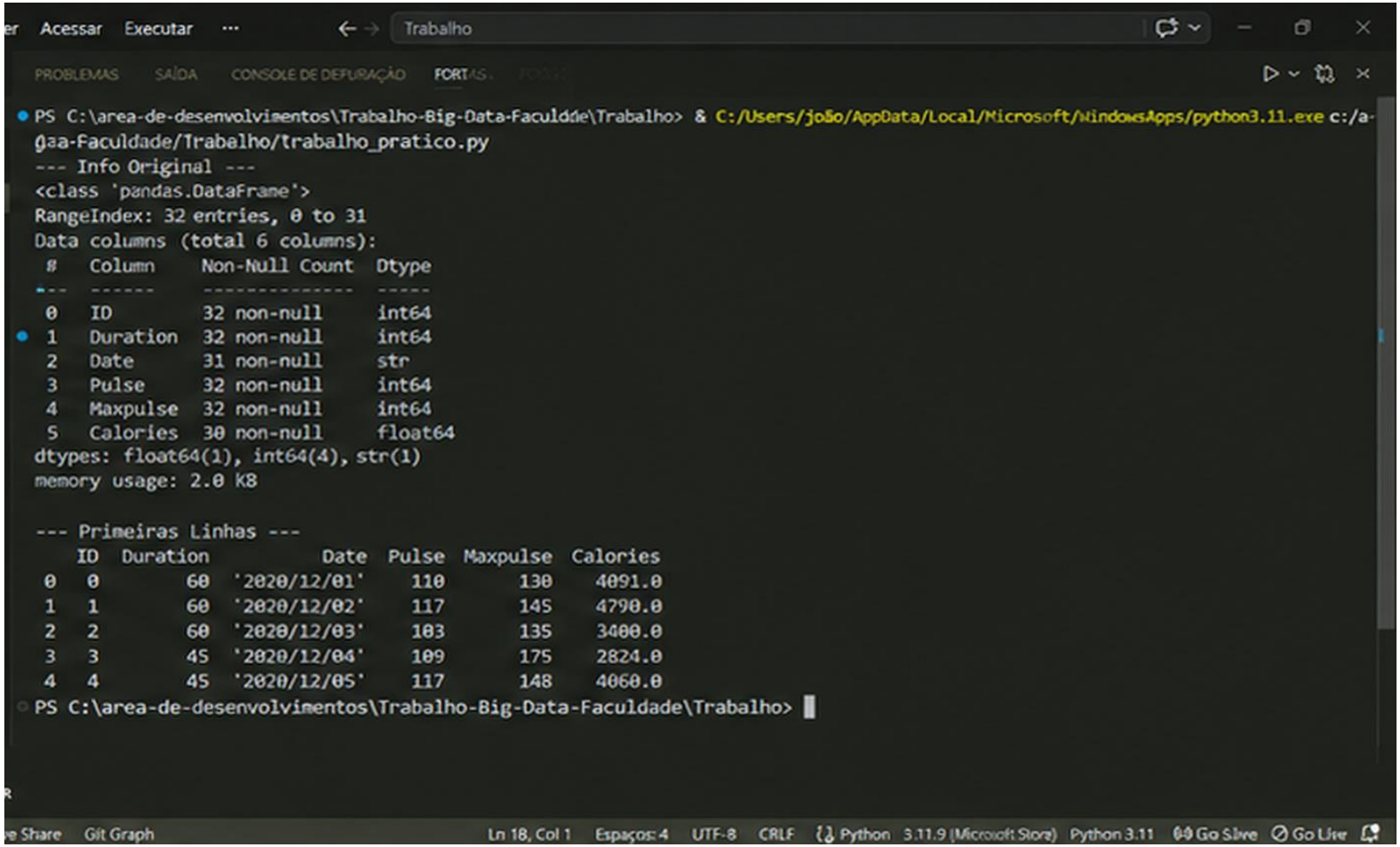
Trabalho Prático | Passo 1 ao 5: Preparação e Cópia de Segurança

import pandas as pd

1. Ler o CSV (usando as configurações que já validamos) df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

2. Verificar a importação print("--- Info Original ---") df.info() print("\n--- Primeiras Linhas ---") print(df.head())

Criar uma variável com a CÓPIA dos dados df_limpo = df.copy()



Explicação: Primeiro, carregamos os dados e criamos uma cópia. a cópia é uma boa prática para nunca perder os dados originais se algo der errado na limpeza.

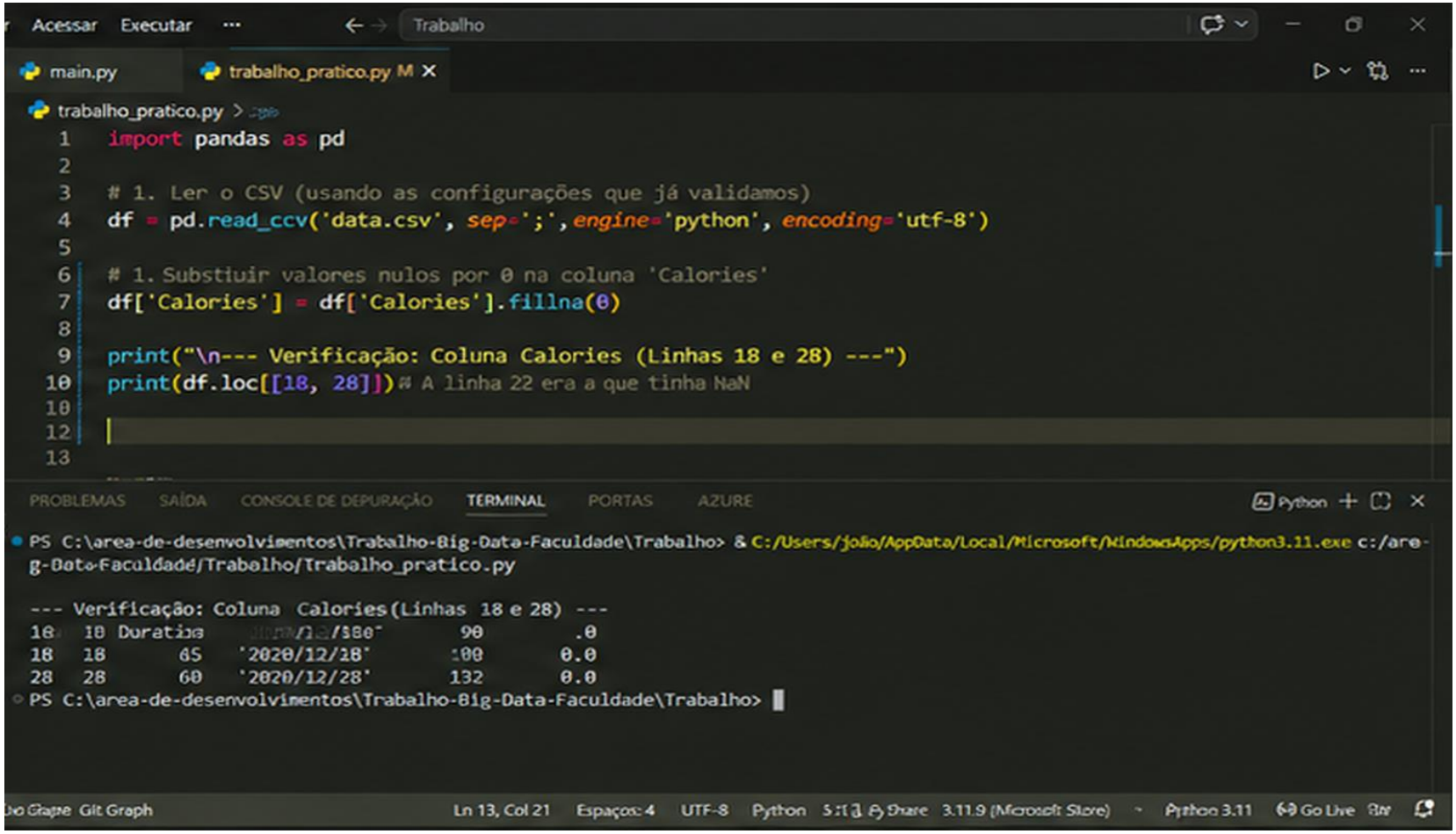
Trabalho Prático| Passo 6: Tratando a coluna ‘Calories’

import pandas as pd

1. Ler o CSV (usando as configurações que já validamos) df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

Substituir valores nulos por 0 na coluna 'Calories' df['Calories'] = df['Calories'].fillna(0)

print("\n--- Verificação: Coluna Calories (Linhas 18 e 28) ---") print(df.loc[[18, 28]])



Explicação : As linhas 18 e 28 possuem valores nulos (NaN). Vamos assumir que, para este estudo, onde não há registro de calorias, o valor será zero

Trabalho Prático | Passo 7: O desafio da coluna 'Date'

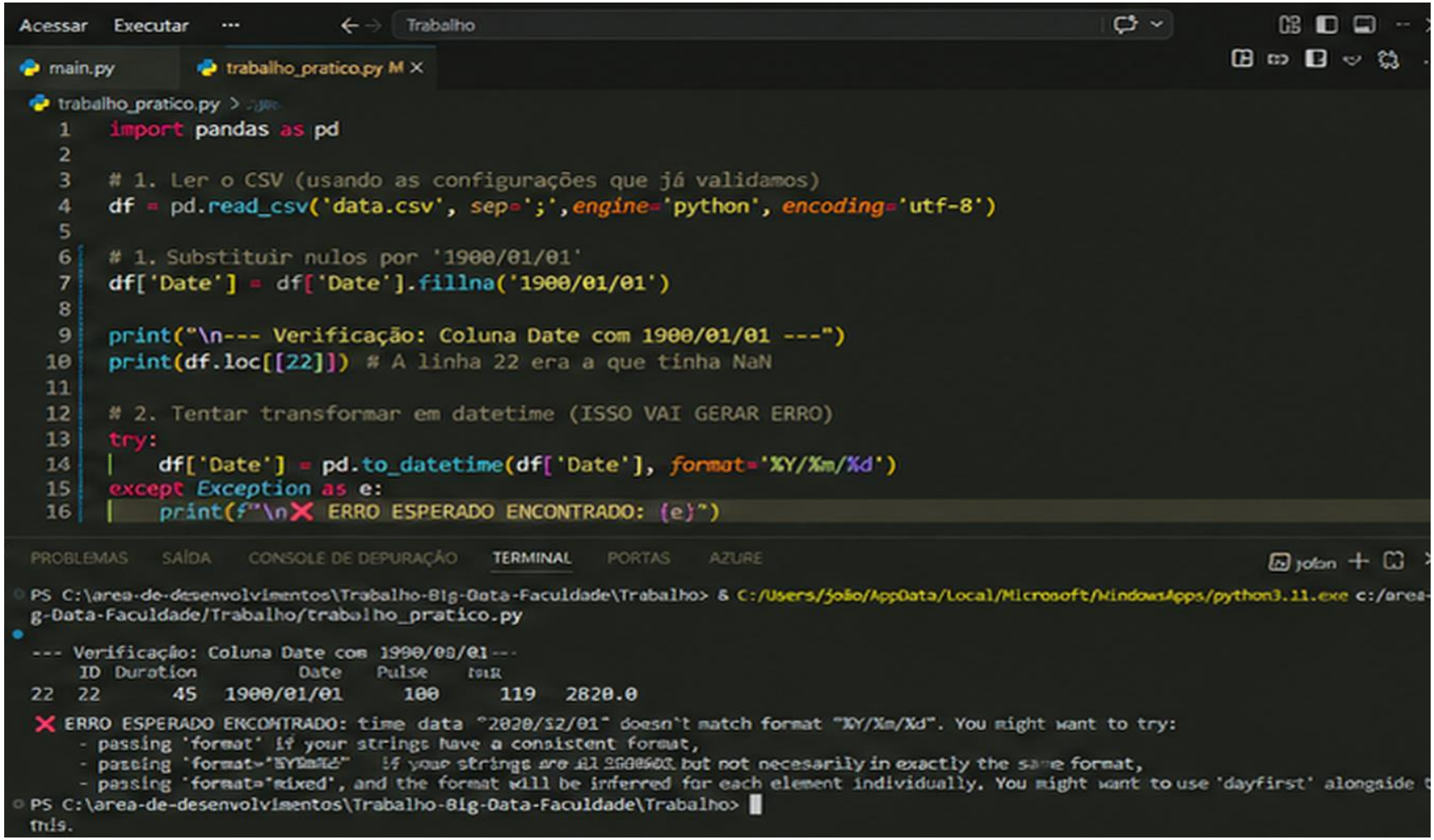
import pandas as pd

1. Ler o CSV (usando as configurações que já validamos) df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

1. Substituir nulos por '1900/01/01' df['Date'] = df['Date'].fillna('1900/01/01')

print("\n--- Verificação: Coluna Date com 1900/01/01 ---") print(df.loc[[22]]) # A linha 22 era a que tinha NaN

2. Tentar transformar em datetime (**isso vai gerar erro**) try: df['Date'] = pd.to_datetime(df['Date'], format='%Y/%m/%d') except Exception as e: print(f"\n ERRO ESPERADO ENCONTRADO: {e}")



Explicação: O Pandas é rigoroso. Se você notar o arquivo original, as datas estão entre aspas simples (ex: '2020/12/01'). Quando inserimos 1900/01/01 sem as aspas ou com um formato que ele não espera dentro daquela mistura de strings, ele trava para garantir a integridade.

Trabalho Prático | Passo 8 ao 11: Corrigindo as Datas e Formatos

import pandas as pd

import numpy as np

1. Ler o CSV (usando as configurações que já validamos) df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

8. Substituir o valor '1900/01/01' por NaN (Not a Number)

df_limpo = df.copy()

df_limpo['Date'] = (

df_limpo['Date']

.astype('string') .

str.strip()

.str.strip("'")

.str.strip('"'))

df_limpo['Date'] = df_limpo['Date'].replace('1900/01/01', pd.NA)

9. Corrigir o erro do valor "20201226" (Linha 26)

Primeiro usamos o replace para ajustar a string e depois tentamos a conversão df_limpo['Date'] = df_limpo['Date'].replace('20201226', '2020/12/26')

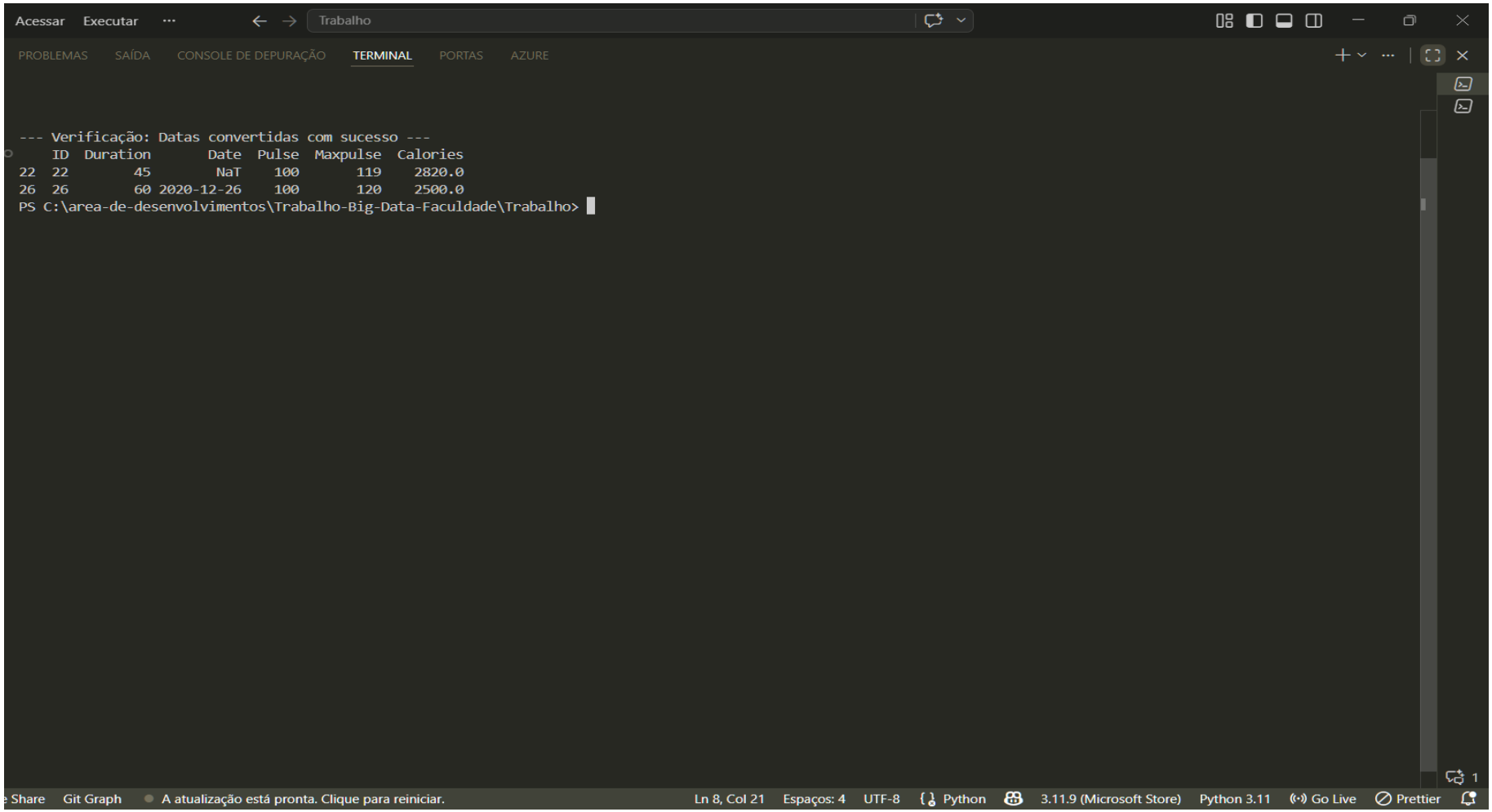
10. Agora convertemos toda a coluna para o formato Datetime

O parâmetro errors='coerce' transformaria erros em NaN,

mas como o roteiro pede para tratar especificamente, usamos a conversão direta:

df_limpo['Date'] = pd.to_datetime(df_limpo['Date'], format='%Y/%m/%d', errors='coerce')

print("\n--- Verificação: Datas convertidas com sucesso ---") print(df_limpo.loc[[22, 26]]) # Verificando a linha 22 (que virou NaN) e a 26 (ajustada)



Explicação : Tratar o erro do valor 1900/01/01 e o formato numérico da linha 26 (20201226).

Trabalho Prático | Passo 12: Removendo registros nulos

import pandas as pd

import numpy as np

1. Ler o CSV (usando as configurações que já validamos) df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

11. Remover registros onde a coluna 'Date' possui valores nulos

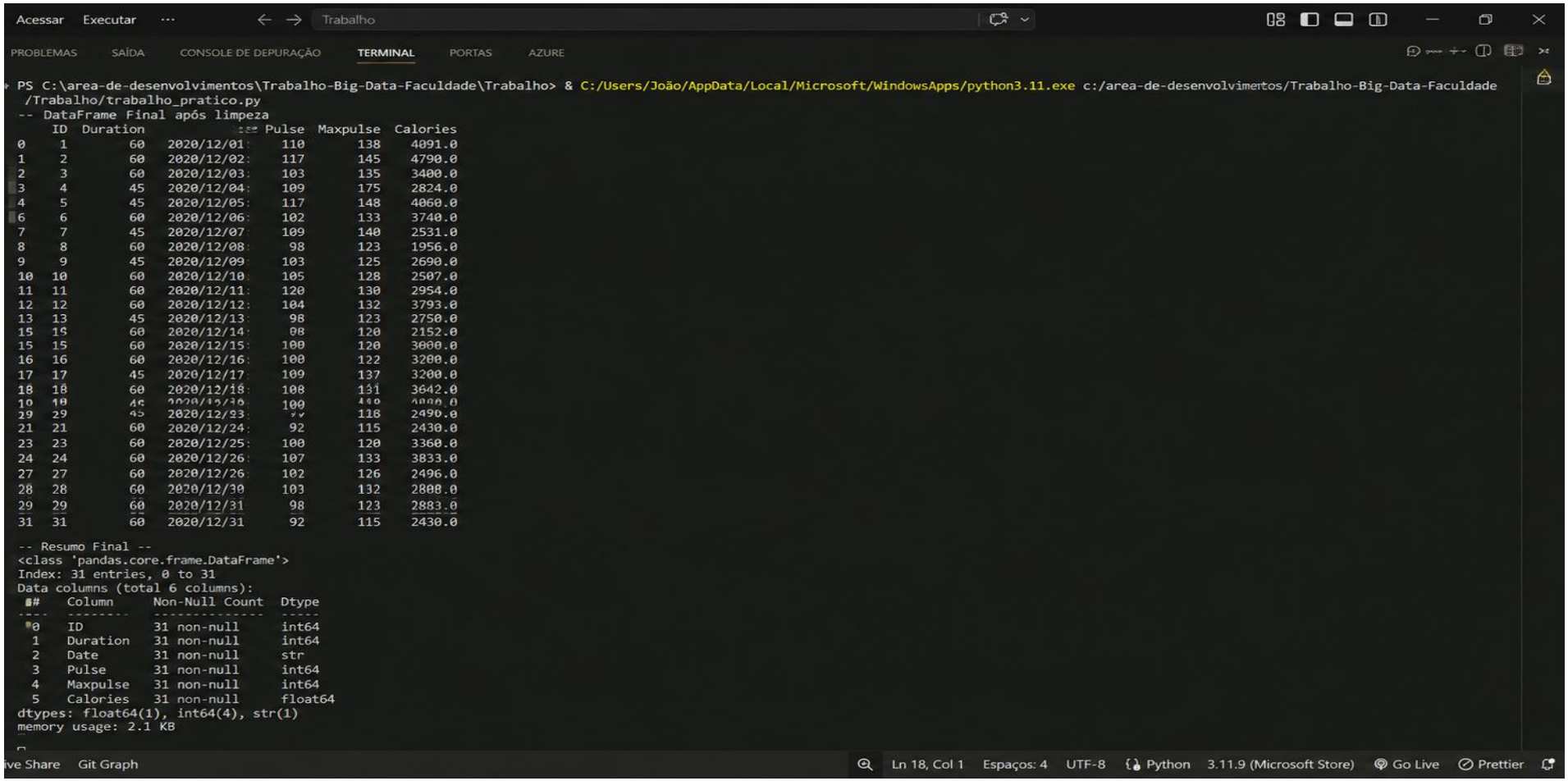
O subset garante que ele olhe apenas para a coluna Date para decidir a exclusão

df_limpo = df.copy()

df_limpo.dropna(subset=['Date'], inplace=True)

12. Imprimir o resultado final para validação print("\n--- DataFrame Final após Limpeza ---") print(df_limpo.to_string())

Verificação final de informações print("\n--- Resumo Final ---") df_limpo.info()



Explicação : O objetivo final de uma limpeza muitas vezes é descartar o que sobrou de "lixo" ou dados incompletos que podem enviesar a análise. Como a linha 22 não tem data válida, vou removê-la.

Link do repositório do GitHub

https://github.com/jotacodeq/Trabalho-Pr-tico-DGT2823-Tecnologias-para-desenv.-de-solu-es-de-big-data.git